

RAG 技术详解与实战应用

第7讲：检索升级实践：

亲手打造“更聪明”的文档理解系统！



目录



1. 上节回顾



2. RAG效果评测



3. 提升RAG系统的召回率



4. 综合优化策略的RAG



RAG效果评测

- **关键环节：** RAG系统的效果取决于召回相关性、上下文有效性、生成忠诚度和答案相关性四个关键环节。优化这些环节可以系统性提升RAG的整体表现。
- **评估指标：** 通过召回率、上下文相关性、忠诚度和答案相关性来评估RAG系统的性能。高召回率和上下文相关性有助于生成更贴合用户问题的答案。
- **综合评估：** 单一指标可能误导系统效果评估，需综合考虑多个指标，以确保系统输出更可信的答案。

提升RAG系统的召回率

- **查询重写策略：** 通过扩写查询、子问题查询和多步骤查询等方法，提升检索准确率和召回率。这些策略有助于系统更好理解用户意图，提升检索效果。
- **优化检索策略：** 通过节点组构造、向量化表示和相似度计算等方法，优化检索过程。合理组合这些策略可以显著提升召回质量，为生成环节提供更相关的上下文。

多路召回的RAG

- **多路召回机制：** 融合多种检索方式，结合重排序策略提高生成质量。通过多个检索器处理不同来源或粒度的信息，再融合排序，能构建更完整上下文，优化整体效果。
- **应用场景：** 多路召回RAG适用于处理复杂查询、覆盖更广内容，并增强答案的完整性与准确性，尤其适用于多领域或信息密集型场景。



目录



1. 上节回顾



2. RAG效果评测



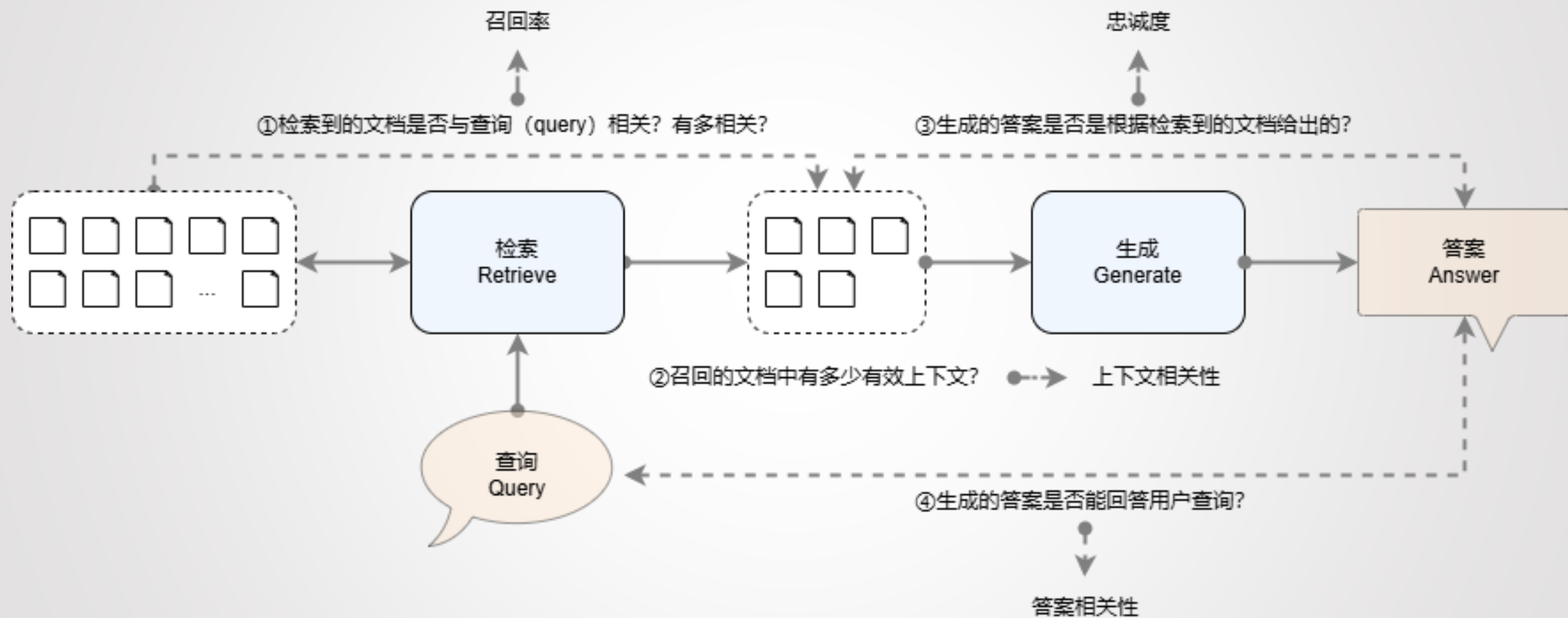
3. 提升RAG系统的召回率



4. 综合优化策略的RAG



RAG效果评测



检索组件的评测指标主要包括两个：

上下文召回率 (Context Recall)、上下文相关性 (Context Relevance)

上下文召回率

- **目标：**评估检索内容的全面程度

计算方式：

- 人工标注相关上下文集合： *Creferance*
- 检索返回上下文集合： *Cretrieved*

公式：

$$R = \frac{\text{总相关上下文数量} \cap \text{召回的上下文数量}}{\text{总相关上下文数量}} \\ = |Creferance \cap Cretrieved| / Creferance$$

指标意义：

- 值越高，信息越全

上下文相关性

- **目标：**评估检索内容与查询的语义匹配程度

计算方式：

- 标注相关句子集合： *Sreferance* (基于人工标注的上下文)
- 检索返回的句子集合： *Sretrieved*

公式：

$$CR = \frac{\text{总相关句子数量} \cap \text{召回的句子数量}}{\text{总相关句子数量}} \\ = |Sreferance \cap Sretrieved| / Sretrieved$$

指标意义：

- 值越高，噪声越少，语义相关性越强。



检索组件评测示例



✓ **LazyLLM 提供了上述两种评估方法的调用接口，通过传入召回文档和预期文档即可得到评估结果，代码如下：**

准备评估数据样例

```
1 data = [{'question': '非洲的猴面包树果实的长度约是多少厘米? ',  
2         'answer': '非洲猴面包树的果实长约15至20厘米。',  
3         'context_retrieved': ['非洲猴面包树是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲，它的果实长约15至20厘米。',  
4         '钙含量比菠菜高50%以上，含较高的抗氧化成分。'],  
5         'context_reference': ['非洲猴面包树是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲，它的果实长约15至20厘米。']  
6 ]]
```

初始化非LLM召回率评估器

```
7 m_recall = NonLLMContextRecall()  
8 res = m_recall(data) # 1.0
```

初始化基于LLM的召回率评估器

```
9 m_lcr = LLMContextRecall(lazyllm.OnlineChatModule())  
10 res = m_lcr(data)
```

初始化上下文相关性评估器

```
11 m_cr = ContextRelevance()  
12 res = m_cr(data) # 0.5
```



检索组件评测示例 – 视频



资源管理器

COURSE [SSH: TEST0224]

LazyRAG-Mineru

rag_pdf_data

src

.env

.gitignore

README.md

requirements.txt

rag_data

Tutorial

rag

codes

chapter1

chapter2

chapter3

chapter5

chapter7

chapter8

chapter9

chapter10

chapter12

chapter14

chapter15

chapter17

chapter18

courseware

courseware_codes

chapter5

chapter7

rag_eval.py 2, U

lazyllm_mineru

LazyRAG-Mineru

README.md

notebook

README.md

.flake8

LICENSE

README.md

test.py

大纲

时间线

rag_eval.py 2, U X

Tutorial > rag > courseware_codes > chapter7 > rag_eval.py > ...

```
1 import lazyllm
2 from lazyllm.tools.eval import LLMContextRecall, NonLLMContextRecall, ContextRelevance
3
4 # 检索组件要求准备满足如下格式要求的数据进行评估
5 data = [{'question': '非洲的猴面包树果实的长度约是多少厘米? ',
6          # 当使用基于LLM的评估方法时要求答案是标注的正确答案
7          'answer': '非洲猴面包树的果实长约15至20厘米。',
8          # context_retrieved 为召回器召回的文档，按段落输入为列表
9          'context_retrieved': ['非洲猴面包树是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲。它的果实长约15至20厘米。',
10                               '钙含量比菠菜高50%以上，含较高的抗氧化成分。'],
11          # context_reference 为标注的应当被召回的段落
12          'context_reference': ['非洲猴面包树是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲。它的果实长约15至20厘米。']}
13
14 # 返回召回文档的命中率，例如上述data成功召回了标注的段落，因此召回率为1
15 m_recall = NonLLMContextRecall()
16 print(m_recall(data)) # 1.0
17
18 # 返回召回文档中的上下文相关性分数，例如上述data召回的两个句子中只有一个是相关的
19 m_cr = ContextRelevance()
20 print(m_cr(data)) # 0.5
21
22 # 返回基于LLM计算的召回率，LLM基于answer和context_retrieved判断是否召回了所有相关文档
23 # 适用于没有标注的情况下使用，比较耗费 token，请根据需求谨慎使用
24 m_lcr = LLMContextRecall(lazyllm.OnlineChatModule(source='sensenova', model='SenseChat-5'))
25 print(m_lcr(data))
```

问题 2

输出

调试控制台

终端 18

(lazyllm) → chapter7 git:(add-mineru) x

zsh courseware_co...

zsh chapter7

SSH: test0224

add-mineru*

0 2

18

行 12, 列 36

空格: 4

UTF-8

LF

() Python

3.10.9 (lazyllm: conda)



目录



1. 上节回顾



2. RAG效果评测



3. 提升RAG系统的召回率



4. 综合优化策略的RAG



查询重写策略 – 原理回顾

1. 查询扩写

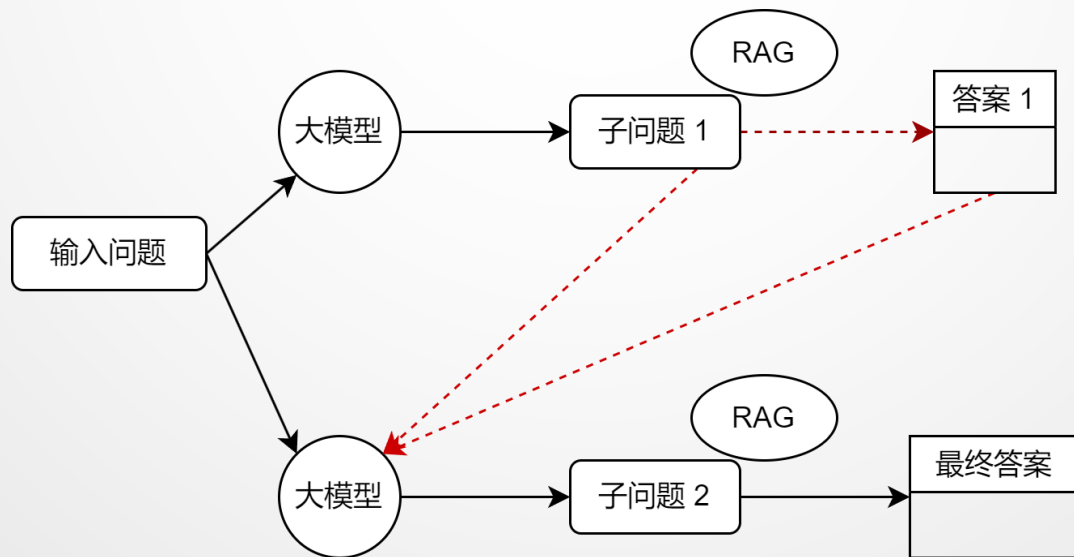
用户查询常常信息不足或存在歧义，影响检索效果。查询扩写通过同义词补充、上下文补全、模板转换等方式，使查询更清晰具体，从而提升准确率和召回率。

2. 子问题查询

将主问题拆解为多个子问题，有助于系统从不同角度理解问题并生成更全面的答案。每个子问题单独检索，最后合并答案。适用于抽象或复合型查询，但需注意处理性能开销。

3. 多步骤查询

将复杂问题拆解为连续的多个步骤，每一步依赖上一步的答案推进。适合需要多轮推理的场景，能有效提升对复杂问题的处理能力。



查询重写策略 - 视频讲解



2_query_rewrite.py — course [SSH: test0224]

2_query_rewrite.py 2 x

courseware_codes > chapter7 > 2_query_rewrite.py > ...

```
1 import lazyllm
2 from lazyllm import Document, ChatPrompter, Retriever
3
4 # rewrite_prompt1 = "你是一个查询改写写助手，将用户的查询改写的更加清晰。\\
5 #     注意，你不需要对问题进行回答，只需要对原始问题进行改写。\\
6 #     下面是一个简单的例子：\\
7 #     输入：RAG\\
8 #     输出：为我介绍下RAG。\\
9 #     \\
10 #     用户输入为："
11
12 robot_prompt = '你是一个友好的 AI 问答助手，你需要根据给定的上下文和问题提供答案。\\
13 | | | 根据以下资料回答问题：\\
14 | | | {context_str} \\n '
15
16 # 加载文档，定义检索器在线大模型。
17 documents = Document(dataset_path="/mnt/lustre/share_data/dist/cmrc2018/data_kb") # 请在 dataset_path 传入数据集绝对路径
18 retriever = Retriever(doc=documents, group_name="CoarseChunk", similarity="bm25_chinese", topk=3) # 定义检索组件
19 llm = lazyllm.OnlineChatModule(source='qwen', model='qwen-turbo') # 调用大模型
20
21 llm.prompt(ChatPrompter(instruction=robot_prompt, extra_keys=['context_str']))
22
23 # 推理
24 query = "MIT OpenCourseWare"
25 # 查询改写
26 # query_rewriter = llm.share(ChatPrompter(instruction=rewrite_prompt1))
27 # query = query_rewriter(query)
28 # print(f"改写后的查询: \\n{query}")
29
30 # 将Retriever组件召回的节点全部存储到列表doc_node_list中
31 doc_node_list = retriever(query=query)
32
33 # 将query和召回节点中的内容组成dict，作为大模型的输入
34 res = llm({"query": query, "context_str": "".join([node.get_content() for node in doc_node_list])})
35
36 print(f'回答: \\n{res}')
```

问题 2 输出 调试控制台 终端 端口 22

o (lazyllm) → chapter7



SSH: test0224 0 2 22

行 17, 列 95 空格: 4 UTF-8 LF Python 3.10.9 ('lazyllm': conda)



可以用下面这个公式进行概括LazyLLM中的召回器 (Retriever)

$$\text{Retriever} = \text{Node Group} \times \text{Similarity} \times \text{Index}$$

01 被检索对象的切分或构造规则 (节点组, Node Group)

02 被检索对象的表征方式 (是否向量化)

03 被检索对象的和问题的相似性计算方式 (Similarity)

04 在指定存储上面的索引策略 (Index)



精度

速度

其中, Node Group 决定了检索的最小单元, Similarity 控制了相似度判断方式, Index 则决定了检索的底层效率与机制。三者相互协同, 共同影响最终的召回表现。



优化检索策略 – 原理回顾



可以用下面这个公式进行概括LazyLLM中的召回器 (Retriever)

$$\text{Retriever} = \text{Node Group} \times \text{Similarity} \times \text{Index}$$

NodeGroup

原文

段落

句子

关键词

总结

Similarity

向量

cosine

dot

bm25

非向量

Index

线性

字典树

KNN

ANN

...



优化检索策略 – LazyLLM的节点组



LazyLLM文档解析流程回顾

文本解析 (Document Read)

1. 文件读取

- 自动调用对应格式的Reader (如txt/docx/html)

2. 文档解析与清洗, 结构化处理

- 清洗冗余标签 (如HTML标签)
- 输出RAG友好的格式 (聚焦正文内容)

3. 生成原始节点组 (Origin Node Group)

- 每个节点存储完整的原文

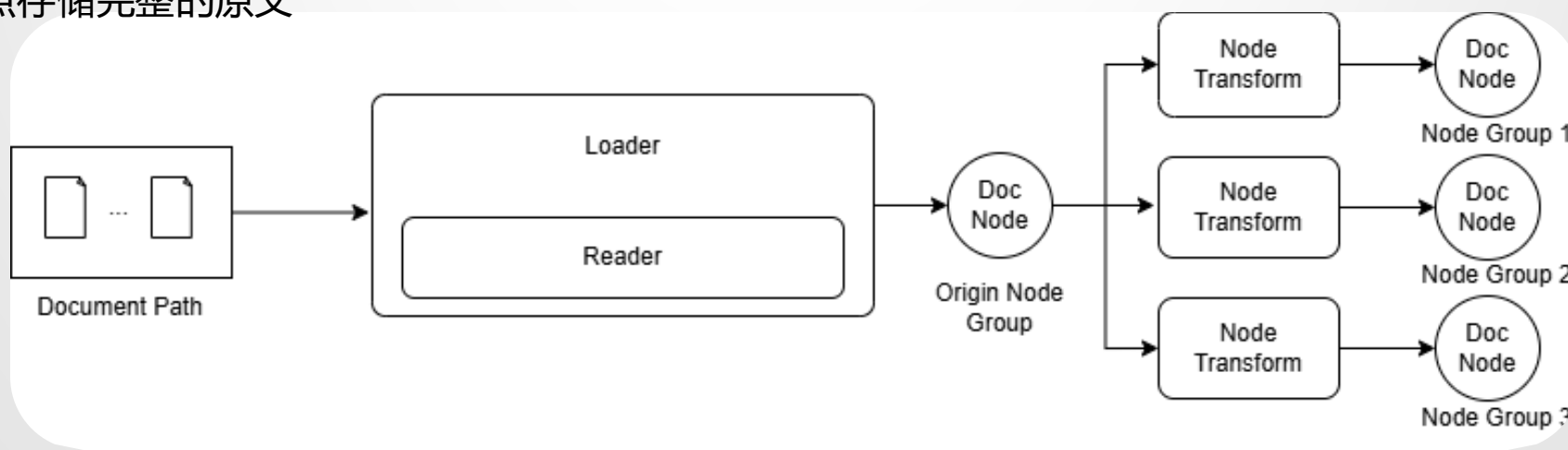
节点转换 (Node Transform)

1. 规则化映射

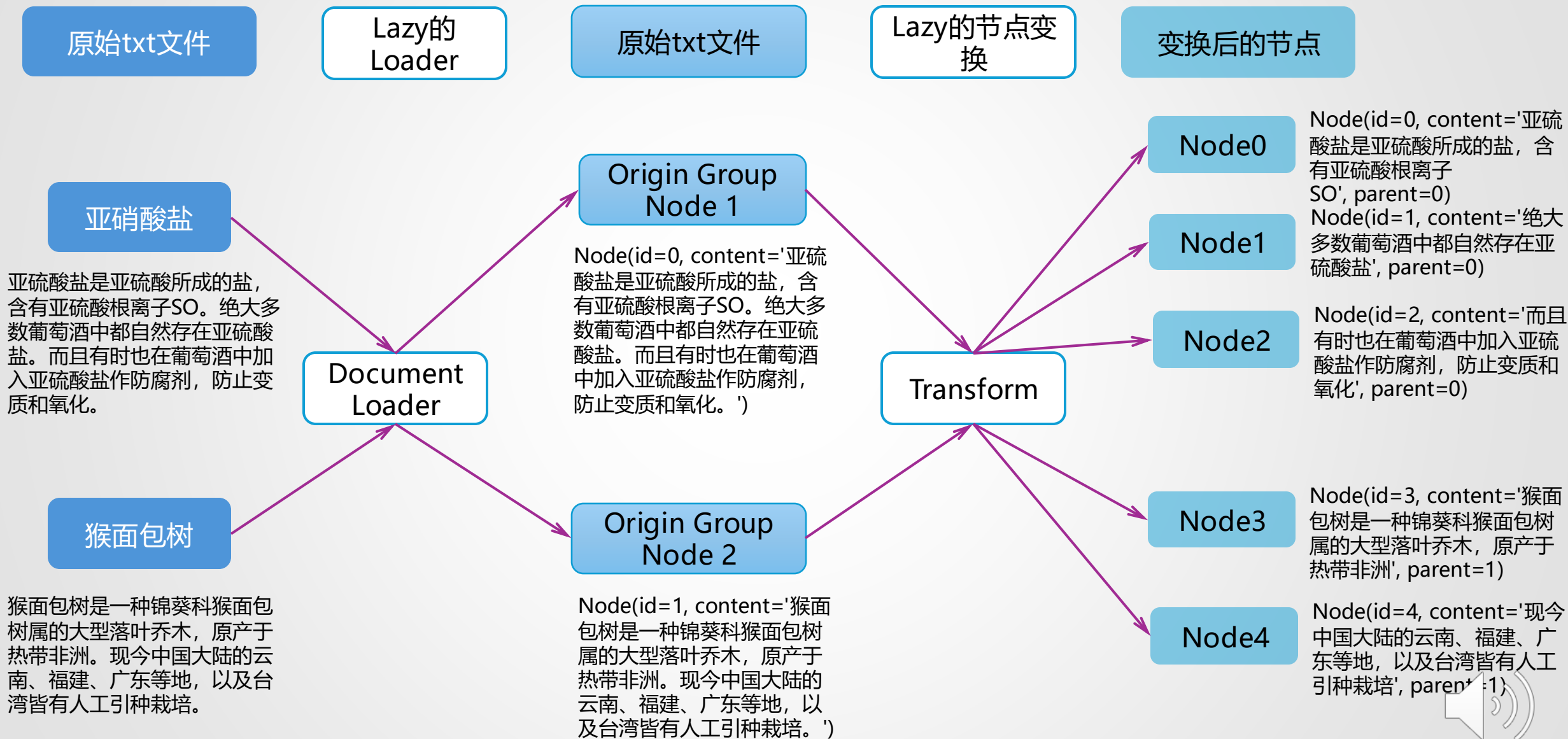
- 单节点→单/多节点转换, 生成新节点组
- 节点内容: 文本片段、摘要等

2. 节点关联信息

- 父子节点关系、元数据 (文档来源等)
- 辅助检索的关键信息



优化检索策略 – LazyLLM的节点组



对于节点组的使用，有以下两种方式：

使用LazyLLM内置的节点组，或创建（注册）新的节点组

节点组的使用

使用预置的节点组：

- CoarseChunk：块大小为 1024，重合长度为 100
- MediumChunk：块大小为 256，重合长度为 25
- FineChunk：块大小为 128，重合长度为 12

创建新的节点组：

通过Document提供的create_node_group函数，传入节点组的名称，转换函数，以及相应的参数，创建新的节点组

节点转换函数

使用内置的节点转换函数：

- SentenceSplitter：输入 chunk_size 和 chunk_overlap 参数实现任意固定长度的分块
- LLMParser：通过大模型对文本进行转换，提取关键词，摘要或者问答对。

自定义新的节点转换函数：

transform 是 Callable 一个对象，推荐的形式有：

- 函数
- 仿函数（定义了 __call__ 方法的类的实例化对象
- 匿名函数（lambda函数）



优化检索策略 – 一个例子



核心流程代码 (标号按执行顺序排列)

```
1. from lazyllm import Document, Retriever # 导入核心模块
2. from lazyllm.tools.rag.doc_node import DocNode # 导入文档节点类

3. docs = Document("/path/to/data") # 初始化文档对象

# ----- 定义分块方法 -----
4. def split_by_sentence1(node: str) -> List[str]: # 字符串分块函数
5.     return node.split('。 ')

6. def split_by_sentence2(node: DocNode) -> List[DocNode]: # DocNode分块函数
7.     return [DocNode(text=t) for t in node.get_text().split('。 ')]

8. class SymbolSplitter: # 可配置分块器类
9.     def __init__(self, splitter="。 ", trans_node=False):
10.         self._splitter = splitter
11.         self._trans_node = trans_node
12.     def __call__(self, node):
13.         return node.split(self._splitter) if not self._trans_node else node.get_text().split(self._splitter)

# ----- 创建节点组 -----
14. docs.create_node_group(name='block1', transform=split_by_sentence1)
15. docs.create_node_group(name='block2', transform=split_by_sentence2, trans_node=True)
16. docs.create_node_group(name='block3', transform=SymbolSplitter())
17. docs.create_node_group(name='block4', transform=SymbolSplitter(trans_node=True), trans_node=True)
18. docs.create_node_group(name='block5', transform=SymbolSplitter(splitter="\n"))
19. docs.create_node_group(name='block6', transform=lambda b: b.split('。 '))

# ----- 检索验证 -----
20. for i in range(6): # 遍历所有分块方式
21.     retriever = Retriever(docs, group_name=f'block{i+1}', similarity="bm25_chinese", topk=1)
22.     print(retriever("亚硫酸盐有什么作用? ")[0].get_content())
```



优化检索策略 – LazyLLM的节点组



diy_transform.py — course [SSH: test0224]

diy_transform.py 2 x

courseware_codes > chapter7 > diy_transform.py > ...

```
1 from typing import List, Union
2 from lazyllm import Document, Retriever
3 from lazyllm.tools.rag.doc_node import DocNode
4
5 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb")
6
7 |
8 # 第一种: 函数实现直接对字符串进行分块规则
9 def split_by_sentence1(node: str, **kwargs) -> List[str]:
10     """函数接收字符串, 返回字符串列表, 输入为trans_node=False时调用"""
11     return node.split('.')
12
13
14 docs.create_node_group(name='block1', transform=split_by_sentence1)
15
16
17 # 第二种: 函数实现获取DocNode对应文本内容后进行分块, 并构造DocNode
18 # 适用于返回非朴素DocNode, 例如LazyLLM提供了ImageDocNode等特殊DocNode
19 def split_by_sentence2(node: DocNode, **kwargs) -> List[DocNode]:
20     """函数接收DocNode, 返回DocNode列表, 输入为trans_node=False时调用"""
21     content = node.get_text()
22     nodes = []
23     for text in content.split('.'):
24         nodes.append(DocNode(text=text))
25     return nodes
26
27
28 docs.create_node_group(name='block2', transform=split_by_sentence2, trans_node=True)
29
30
31 # 第三种: 实现了 __call__ 函数的类
32 # 优点是一个类用于多种分块, 例如这个例子可以通过控制实例化时的参数实现基于多种符号的分块
33 class SymbolSplitter:
34     """实例化后传入Transform, 默认情况下接收字符串, trans_node为true时接收DocNode"""
35     def __init__(self, splitter=".", trans_node=False):
36         self.splitter = splitter
37         self.trans_node = trans_node
38
```

问题 2 输出 调试控制台 终端 端口 18

o (lazyllm) + chapter7 git:(courseware_codes) x

+ zsh - chapter7



优化检索策略 – 特殊节点组

核心流程代码 (标号按执行顺序排列)



```
1. import lazyllm 导入LazyLLM主库

2. llm = lazyllm.OnlineChatModule(source="qwen", model="qwen-turbo") # 初始化在线大模型接口 (需配置API-Key)

3. summary_llm = lazyllm.LLMParser(llm, language="zh", task_type="summary") # 创建摘要解析器
4. keyword_llm = lazyllm.LLMParser(llm, language="zh", task_type="keywords") # 创建关键词解析器
5. qapair_llm = lazyllm.LLMParser(llm, language="zh", task_type="qa") # 创建问答对解析器

6. docs = lazyllm.Document("test_parse") # 初始化文档对象

# 注册摘要节点组
7. docs.create_node_group(name='summary', transform=summary_llm, trans_node=True, parent='CoarseChunk' )

# 注册关键词节点组
8. docs.create_node_group(name='keyword', transform=keyword_llm, trans_node=True, parent='CoarseChunk' )

# 注册问答对节点组
9. docs.create_node_group(name='qapair', transform=qapair_llm, trans_node=True, parent='CoarseChunk' )

10. group_names = ["CoarseChunk", "summary", "keyword", "qapair # 定义待验证的节点组列表
11. for group_name in group_names: # 遍历所有节点组
12.     print(f"===== 正在解析 {group_name}, 首次解析耗时较长, 请耐心等待 =====") # 进度提示
13.     retriever = lazyllm.Retriever(docs, group_name=group_name, similarity="bm25_chinese", topk=1) # 初始化检索器
14.     node = retriever("亚硫酸盐有什么作用? ") # 执行相似度查询
15.     print(node[0].get_content()) # 输出检索结果内容
```



优化检索策略 - 特殊节点组



5_use_llmparser.py x

course > courseware_codes > chapter7 > 5_use_llmparser.py > ...

```
1 import lazyllm
2
3 # 如果您要使用在线大模型, 请将您要使用的在线模型 Api-key 抛出为环境变量
4 llm = lazyllm.OnlineChatModule(source="qwen", model="qwen-turbo") # 调用大模型
5
6 # LLMParser 是 LazyLLM 内置的基于 LLM 进行节点组构造的类, 支持 summary, keywords和qa三种
7 summary_llm = lazyllm.LLMParser(llm, language="zh", task_type="summary") # 摘要提取LLM
8 keyword_llm = lazyllm.LLMParser(llm, language="zh", task_type="keywords") # 关键词提取LLM
9 qapair_llm = lazyllm.LLMParser(llm, language="zh", task_type="qa") # 问答对提取LLM
10
11 docs = lazyllm.Document("test_parse")
12
13 docs.create_node_group(name='summary', transform = summary_llm, trans_node=True, parent='CoarseChunk')
14 docs.create_node_group(name='keyword', transform = keyword_llm, trans_node=True, parent='CoarseChunk')
15 docs.create_node_group(name='qapair', transform = qapair_llm, trans_node=True, parent='CoarseChunk')
16
17 # 查看节点组内容, 此处我们通过一个检索器召回一个节点并打印其中的内容, 后续都通过这种方式实现
18 group_names = ["CoarseChunk", "summary", "keyword", "qapair"]
19 for group_name in group_names:
20     print(f"===== 正在解析 {group_name}, 首次解析耗时较长, 请耐心等待 =====")
21     retriever = lazyllm.Retriever(docs, group_name=group_name, similarity="bm25_chinese", topk=1)
22     node = retriever("亚硫酸盐有什么作用? ")
23     print(node[0].get_content())
24
```

问题 输出 调试控制台 终端 端口 14

o (lazyllm) → chapter7

zsh LazyLLM
zsh chapter7

优化检索策略 – 构造复杂的节点组树

✓ 下面我们将定义一个满足如下结构的 Node Group，通过查看这些节点组节点的具体内容分析不同节点组适用的任务场景。

1. 粗粒度节点组

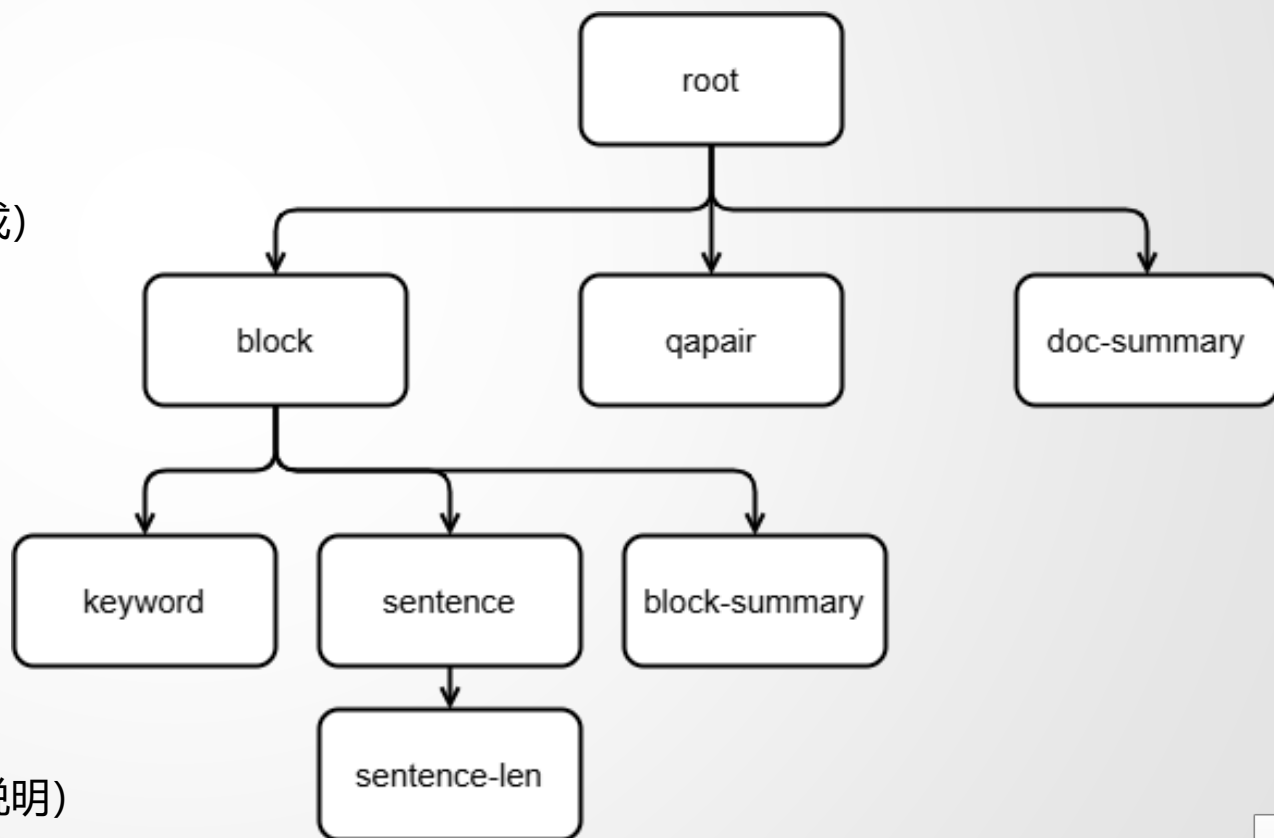
1. **特征：**大段落或章节级拆分
2. **适用场景：**
 1. 查询主题宽泛（如概述性问答）
 2. 需要上下文连贯性的任务（如摘要生成）

2. 细粒度节点组

1. **特征：**句子或短语级拆分
2. **适用场景：**
 1. 精准答案检索（如事实型问答）
 2. 高相关性要求的语义匹配

3. 逻辑规则节点组

1. **特征：**按标题、列表、表格等结构化拆分
2. **适用场景：**
 1. 结构化数据查询（如参数对比、步骤说明）
 2. 多模态文档处理



优化检索策略 - 大小块策略



初始化在线大模型模块 (通义千问)

```
1 llm = lazyllm.OnlineChatModule(source='qwen', model='qwen-turbo')
```

定义问答提示模板

```
2 prompt = '你是一个友好的 AI 问答助手，你需要根据给定的上下文和问题提供答案。根据以下资料回答问题：{context_str} \n '
```

创建大模型问答接口

```
3 robot = llm.prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
```

加载文档并创建分句处理节点组

```
4 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb")
```

```
5 docs.create_node_group(name='sentences', transform=(lambda d: d.split("\n") if d else ""), parent=Document.CoarseChunk)
```

创建基于BM25中文相似度的检索器

```
6 retriever = Retriever(docs, group_name="sentences", similarity="bm25_chinese", topk=3)
```

设置查询问题

```
7 query = "都有谁参加了2008年的奥运会? "
```

执行原始节点检索并打印结果

```
8 doc_node_list = retriever(query=query)
```

```
9 doc_node_res = "".join([node.get_content() for node in doc_node_list])
```

获取并打印父节点内容

```
10 parent_list = [node.parent.get_text() for node in doc_node_list]
```

使用父节点内容生成最终答案

```
11 res = robot({"query": query, "context_str": "".join([node_text for node_text in parent_list])})
```



优化检索策略 - 大小块策略



15_parent_node.py 3 X

courseware_codes > chapter7 > 15_parent_node.py > ...

```
1 import lazyllm
2 from lazyllm import bind
3 from lazyllm import Document, Retriever, TrainableModule
4
5 llm = lazyllm.OnlineChatModule(source='qwen', model='qwen-turbo')
6 prompt = '你是一个友好的 AI 问答助手, 你需要根据给定的上下文和问题提供答案.\
7         根据以下资料回答问题: \
8         {context_str} \n '
9
10 robot = llm.prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
11
12 # 文档加载
13 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb")
14 docs.create_node_group(name='sentences', transform=(lambda d: d.split('\n') if d else ''), parent=Document.CoarseChunk)
15
16 # 定义两个不同的检索器在同一个节点组使用不同相似度方法进行检索
17 retriever = Retriever(docs, group_name="sentences", similarity="bm25_chinese", topk=3)
18
19 # 执行查询
20 query = "都有谁参加了2008年的奥运会?"
21
22 # 原节点检索结果
23 doc_node_list = retriever(query=query)
24 doc_node_res = "".join([node.get_content() for node in doc_node_list])
25 print(f"原节点检索结果: \n{doc_node_res}")
26 print('='*100)
27
28 # 分节点对应结果
```

问题 3

输出

调试控制台

终端

端口 22

+

zsh - chapter7

□

□

...

^

X

o (lazyllm) → chapter7 □



优化检索策略 - 向量化



- ✓ Embedding 是一种将文档映射为一个保留原文语义的高维向量的技术，基于这种高维向量可以高效进行语义检索。
- ✓ LazyLLM 支持调用线上和线下 Embedding 模型，线上模型通过 OnlineEmbeddingModule 调用，线下模型通过 TrainableModule 调用。代码如下：

导入核心库和部署模块

```
1 import lazyllm
```

```
2 from lazyllm import deploy
```

初始化在线文本嵌入模块（通义千问）

```
3 online_embed = lazyllm.OnlineEmbeddingModule(source="qwen")
```

部署本地稠密向量嵌入模型（bge-large-zh-v1.5）

```
4 offline_embed = lazyllm.TrainableModule('bge-large-zh-v1.5').start()
```

部署混合稀疏向量嵌入模型（bge-m3）

使用AutoDeploy方法并指定稀疏编码类型

```
5 offline_sparse_embed = lazyllm.TrainableModule('bge-m3').deploy_method((deploy.AutoDeploy, {'embed_type': 'sparse'})).start()
```

测试不同嵌入模型的输出效果

```
7 print("online embed: ", online_embed("hello world"))
```

```
8 print("offline embed: ", offline_embed("hello world"))
```

```
9 print("offline sparse embed: ", offline_sparse_embed("hello world"))
```



优化检索策略 - 向量化



8_use_embedding.py — course [SSH: test0224]

8_use_embedding.py 2 X

courseware_codes > chapter7 > 8_use_embedding.py > ...

```
1 # 将您的本地模型地址抛出为环境变量或使用绝对路径 export LAZYLLM_MODEL_PATH=/path/to/your/models
2 # 将您的在线模型 API key 抛出为环境变量, 以 sensenova 为例
3 #   export LAZYLLM_SENSENOVA_API_KEY=
4 #   export LAZYLLM_SENSENOVA_SECRET_KEY=
5
6 import lazyllm
7 from lazyllm import deploy
8
9 online_embed = lazyllm.OnlineEmbeddingModule(source="qwen")
10 offline_embed = lazyllm.TrainableModule('bge-large-zh-v1.5').start()
11 offline_sparse_embed = lazyllm.TrainableModule('bge-m3').deploy_method((deploy.AutoDeploy, {'embed_type': 'sparse'})).start()
12 print("online embed: ", online_embed("hello world"))
13 print("offline embed: ", offline_embed("hello world"))
14 print("offline sparse embed: ", offline_sparse_embed("hello world"))
15
```

问题 2 输出 调试控制台 终端 端口 25

○ (lazyllm) → chapter7

+ zsh - chapter7



扩展学习 – 接入其他在线向量模型服务



当LazyLLM提供的向量模型无法满足您的需求时，您也可以**自定义在线Embedding服务接入**：

- 1.若接口格式与现有厂商一致，直接替换embed_url即可
- 2.若格式不一致，需继承OnlineEmbeddingModuleBase实现自定义逻辑

导入基础在线嵌入模块

```
1 from lazyllm.module import OnlineEmbeddingModuleBase
```

自定义在线嵌入模块实现

```
2 class CustomOnlineEmbeddingModule(OnlineEmbeddingModuleBase):
3     def __init__(self, embed_url: str = '<new platform embedding url>',
4                 embed_model_name: str = '<new platform embedding model name>'):
5         super().__init__("<new_platform_name>", embed_url, lazyllm.config['new_platform_api_key'], embed_model_name)
6
7     def _encapsulated_data(self, text:str, **kwargs):
8         json_data = {
9             "input": [text],
10            "model": self._embed_model_name
11        }
12        if len(kwargs) > 0:
13            json_data.update(kwargs)
14        return json_data
15
16     def _parse_response(self, response: dict[str, any]):
17         return response['embeddings'][0]['embedding']
```



优化检索策略 – 向量检索



```
# 导入必要的库模块
1 import lazyllm
2 from lazyllm import Document, Retriever, TrainableModule

# 初始化本地嵌入模型 (bge-large-zh-v1.5)
3 embed_model = TrainableModule("bge-large-zh-v1.5").start()

# 加载文档数据并创建处理节点组
4 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embed_model)
# 创建基于换行符分割的文本块节点组
5 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))

# 创建基于余弦相似度的检索器
6 retriever1 = Retriever(docs, group_name="block", similarity="cosine", topk=3)
# 创建基于BM25算法的检索器 (中文优化版)
7 retriever2 = Retriever(docs, group_name="block", similarity="bm25_chinese", topk=3)

# 定义测试查询语句
8 query = "都有谁参加了2008年的奥运会？"

# 执行双路检索
9 result1 = retriever1(query=query) # 余弦相似度检索
10 result2 = retriever2(query=query) # BM25算法检索

# 输出对比结果
11 print("余弦相似度召回结果：")
12 print("\n\n".join([res.get_content() for res in result1]))
13 print("bm25召回结果：")
14 print("\n\n".join([res.get_content() for res in result2]))
```



优化检索策略 – 向量检索



```
9_cosine_vs_bm25.py — course [SSH: test0224]
9_cosine_vs_bm25.py 2 X
courseware_codes > chapter7 > 9_cosine_vs_bm25.py > ...
1 # 探讨相似度对召回率的影响
2 # 请将您的在线嵌入模型api-key抛出为环境变量或更改为本地模型
3 import lazyllm
4 from lazyllm import Document, Retriever, TrainableModule
5
6 # 定义 embedding 模型
7 embed_model = TrainableModule("bge-large-zh-v1.5").start()
8
9 # 文档加载
10 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embed_model)
11 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
12
13 # 定义两个不同的检索器在同一个节点组使用不同相似度方法进行检索
14 retriever1 = Retriever(docs, group_name="block", similarity="cosine", topk=3)
15 retriever2 = Retriever(docs, group_name="block", similarity="bm25_chinese", topk=3)
16
17 # 执行查询
18 query = "都有谁参加了2008年的奥运会?"
19 result1 = retriever1(query=query)
20 result2 = retriever2(query=query)
21
22 print("余弦相似度召回结果: ")
23 print("\n\n".join([res.get_content() for res in result1]))
24 print("bm25召回结果:")
25 print("\n\n".join([res.get_content() for res in result2]))
26
```

问题 2 输出 调试控制台 终端 端口

zsh - chapter7

(lazyllm) → chapter7

SSH: test0224 courseware_codes* 0 0 2 0

行 15, 列 84 空格: 4 UTF-8 LF Python 3.10.9 ('lazyllm': conda)



优化检索策略 – 向量检索对比结果



指定嵌入模型后，我们就可以通过向量相似度进行文档召回了。LazyLLM.Retriever 可传入的值有：“cosine”，“bm25”和“bm25_chinese”，接下来让我么办比较下“cosine”和“bm25_chinese”的召回效果：

	余弦相似度召回结果	bm25召回结果
1	...参加2008年北京夏季奥林匹克运动会的中国花样游泳队一行共计15人，其中运动员9人，教练4人...	...1996年和2000年夏季奥林匹克运动会两度获得举重项目金牌，为中国奥运史上连续两届获得举重冠军的第一人...参加了亚特兰大奥运会的70公斤级举重比赛...成为该届奥运会的经典一幕。悉尼奥运会后...第三次参加奥运会...
2	2008年夏季奥林匹克运动会男子100米跑赛事在2008年8月15日及8月16日在北京国家体育馆举行...	2008年8月10日，谭望嵩在北京奥运会中国队与比利时队的男足小组赛当中...
3	参加2008年北京夏季奥林匹克运动会的中国皮划艇队共计40人，其中运动员23人，官员与工作人员17人...	...参加2008年北京夏季奥林匹克运动会的中国花样游泳队一行共计15人，其中运动员9人，教练4人...

以上是二者的召回结果：

- ✓ 从查询特点来看，余弦相似度的检索结果同时包含了“2008年”和“奥运会”这两个比较重要的主题。
- ✓ bm25的结果虽然每条结果都包含“奥运会”的信息，但其中有一个结果并没有提及2008年奥运会。



优化检索策略 – 向量检索对比结果



指定嵌入模型后，我们就可以通过向量相似度进行文档召回了。LazyLLM.Retriever 可传入的值有：“cosine”，“bm25” 和 “bm25_chinese”，接下来让我么办比较下 “cosine” 和 “bm25_chinese” 的召回效果：

	余弦相似度召回结果	bm25召回结果
1	...参加2008年北京夏季奥林匹克运动会的中国花样游泳队一行共计15人，其中运动员9人，教练4人...	...1996年和2000年夏季奥林匹克运动会两度获得举重项目金牌，为中国奥运史上连续两届获得举重冠军的第一人...参加了亚特兰大奥运会的70公斤级举重比赛...成为该届奥运会的经典一幕。悉尼奥运会后...第三次参加奥运会...
2	2008年夏季奥林匹克运动会男子100米跑赛事在2008年8月15日及8月16日在北京国家体育馆举行...	2008年8月10日，谭望嵩在北京奥运会中国队与比利时队的男足小组赛当中...
3	参加2008年北京夏季奥林匹克运动会的中国皮划艇队共计40人，其中运动员23人，官员与工作人员17人...	...参加2008年北京夏季奥林匹克运动会的中国花样游泳队一行共计15人，其中运动员9人，教练4人...

以上是二者的召回结果：

- ✓ 从查询特点来看，余弦相似度的检索结果同时包含了“2008年”和“奥运会”这两个比较重要的主题。
- ✓ bm25的结果虽然每条结果都包含“奥运会”的信息，但其中有一个结果并没有提及2008年奥运会。



优化检索策略 - 多Embedding召回



初始化多个嵌入模型

```
1 bge_m3_embed = TrainableModule('bge-m3').start()
2 bge_large_embed = TrainableModule('bge-large-zh-v1.5').start()
3 embeds = {'vec1': bge_m3_embed, 'vec2': bge_large_embed}
```

加载文档并创建处理节点

```
4 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embeds)
5 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
```

创建多embedding检索器 (使用vec1和vec2)

```
6 retriever1 = Retriever(docs, group_name="block", embed_keys=['vec1', 'vec2'], similarity="cosine", topk=3)
```

创建单embedding检索器 (仅使用vec2)

```
7 retriever2 = Retriever(docs, group_name="block", embed_keys=['vec2'], similarity="bm25", topk=3)
```

执行检索并输出结果

```
8 query = "都有谁参加了2008年的奥运会? "
9 result1 = retriever1(query=query)
10 result2 = retriever2(query=query)
11 print( "使用bge-m3和bge-large-zh-v1.5进行余弦相似度召回结果: ")
12 print("\n\n".join([res.get_content() for res in result1]))
13 print("使用bge-large-zh-v1.5进行bm25相似度召回结果: ")
14 print("\n\n".join([res.get_content() for res in result2]))
```



优化检索策略 - 多Embedding召回



5_use_llm_parser.py

10_use_multi_embed.py ×

course > courseware_codes > chapter7 > 10_use_multi_embed.py > ...

```
1 # 探讨不同embedding对召回率的影响
2 # 请将您的在线嵌入模型api-key抛出为环境变量或使用本地模型
3
4 from lazyllm import Document, Retriever, TrainableModule
5
6 # 定义多个 embedding 模型
7 bge_m3_embed = TrainableModule('bge-m3').start()
8 bge_large_embed = TrainableModule('bge-large-zh-v1.5').start()
9 embeds = {'vec1': bge_m3_embed, 'vec2': bge_large_embed}
10
11 # 文档加载
12 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embeds)
13 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
14
15 # 定义两个不同的检索器分别对相同节点组的不同 embedding 进行检索
16 retriever1 = Retriever(docs, group_name="block", embed_keys=['vec1', 'vec2'], similarity="cosine", topk=3)
17 retriever2 = Retriever(docs, group_name="block", embed_keys=['vec2'], similarity="bm25", topk=3)
18
19 # 执行检索
20 query = "都有谁参加了2008年的奥运会?"
21 result1 = retriever1(query=query)
22 result2 = retriever2(query=query)
23
24 print("使用bge-m3和bge-large-zh-v1.5进行余弦相似度召回结果:")
25 print("\n\n".join([res.get_content() for res in result1]))
26 print("使用bge-large-zh-v1.5进行余弦相似度召回结果:")
27 print("\n\n".join([res.get_content() for res in result2]))
28
```

问题

输出

调试控制台

终端

端口 14

o (lazyllm) → chapter7

zsh LazyLLM

zsh chapter7

优化检索策略 -使用相似度阈值过滤



初始化嵌入模型

```
1 bge_embed = TrainableModule('bge-large-zh-v1.5').start()
```

加载文档并创建处理节点

```
2 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=bge_embed)
```

```
3 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
```

创建无阈值过滤的检索器

```
4 retriever1 = Retriever(docs, group_name="block", similarity="cosine", topk=6)
```

创建带阈值过滤的检索器(相似度>0.6)

```
5 retriever2 = Retriever(docs, group_name="block", similarity="cosine", similarity_cut_off=0.6, topk=6)
```

执行检索并输出结果

```
6 query = "2008年参加花样游泳的运动员都有谁? "
```

```
7 result1 = retriever1(query=query)
```

```
8 result2 = retriever2(query=query)
```

```
9 print("未设定 similarity_cut_off: ")
```

```
10 print("\n\n".join([res.get_content() for res in result1]))
```

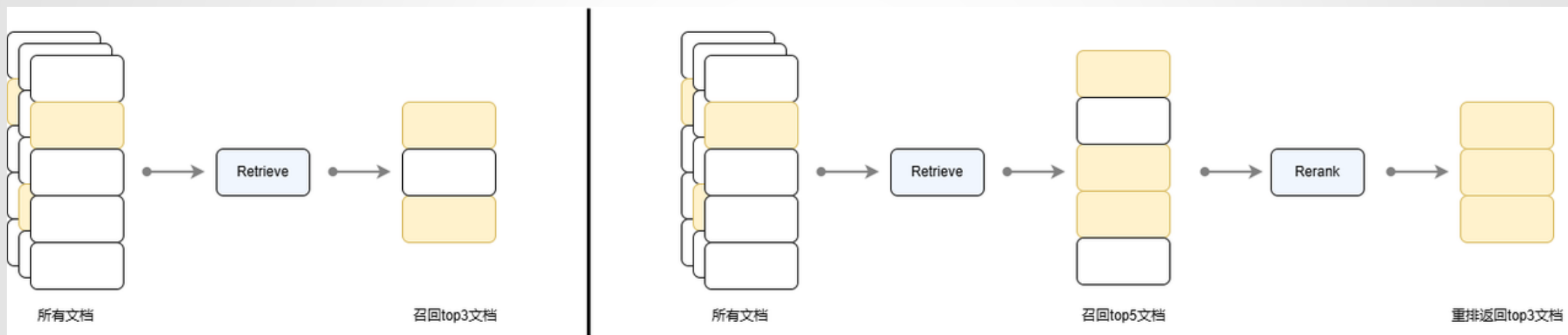
```
11 print("设定 similarity_cut_off=0.6 : ")
```

```
12 print("\n\n".join([res.get_content() for res in result2]))
```



优化检索策略 – 重排序

重排序模型用于两阶段检索的第二阶段，在 LazyLLM 中对应 Reranker 组件。其核心思想是在初步检索得到候选结果后，利用更精细的模型对结果重新排序，从而提升最终效果。



LazyLLM 提供了 Reranker 组件进行重排序，分别提供了在线和离线两种重排序模型的调用途径，其中在线模型通过 `OnlineEmbeddingModule(type="rerank")` 进行调用，离线重排模型则仍然通过 `TrainableModule` 进行调用。



优化检索策略 – 重排序



导入必要的库模块

```
1 from lazyllm import Document, Retriever, Reranker, OnlineEmbeddingModule, TrainableModule
```

初始化本地嵌入模型

```
2 embedding_model = TrainableModule('bge-large-zh-v1.5').start()
```

初始化本地重排序模型

```
3 offline_rerank = TrainableModule('bge-reranker-large').start()
```

加载文档并创建处理节点

```
4 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embedding_model)
```

```
5 docs.create_node_group(name='block', transform=(lambda d: d.split("\n")))
```

创建基础检索器

```
6 retriever = Retriever(docs, group_name="block", similarity="cosine", topk=3)
```

创建重排序器 (输出合并后的文本内容)

```
7 reranker2 = Reranker('ModuleReranker', model=offline_rerank, topk=3, output_format='content', join=True)
```

执行检索和重排序

```
8 query = "猴面包树有哪些功效? "
```

```
9 result1 = retriever(query=query)
```

```
10 result2 = reranker2(result1, query=query)
```

输出结果

```
11 print("余弦相似度召回结果: ")
```

```
12 print("\n\n".join([res.get_content() for res in result1]))
```

```
13 print("开源重排序模型结果: ")
```

```
14 print(result2)
```



优化检索策略 - 重排序



12_use_reranker.py ×

course > courseware_codes > chapter7 > 12_use_reranker.py > ...

```
1 from lazyllm import Document, Retriever, Reranker, TrainableModule
2
3 # 定义嵌入模型和重排序模型
4 embedding_model = TrainableModule('bge-large-zh-v1.5').start()
5
6 # 如果您要使用在线重排模型
7 # 目前LazyLLM仅支持 qwen和glm 在线重排模型, 请指定相应的 API key。
8 # online_rerank = OnlineEmbeddingModule(type="rerank")
9 # 本地重排序模型
10 offline_rerank = TrainableModule('bge-reranker-large').start()
11
12 docs = Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embedding_model)
13 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
14
15 # 定义检索器
16 retriever = Retriever(docs, group_name="block", similarity="cosine", topk=3)
17
18 # 定义重排器
19 # 指定 reranker 的输出为字符串, 并进行串联, 未进行串联时输出为字符串列表
20 reranker = Reranker('ModuleReranker', model=offline_rerank, topk=3, output_format='content', join=True)
21
22 # 执行推理
23 query = "猴面包树有哪些功效?"
24 result1 = retriever(query=query)
25 result2 = reranker(result1, query=query)
26
```

(parameter) output_format: str | None

问题 输出 调试控制台 终端 端口 18

o (lazyllm) → chapter7

zsh LazyLLM

zsh chapter7

优化检索策略 – 重排序

下表是针对查询 “猴面包树有哪些功效？” 的召回结果：

	余弦相似度召回结果	重排结果
1	假苹婆（学名：" "），别称七姐果、鸡冠皮、鸡冠木、山羊角、红郎伞、赛苹婆及山木棉等，为梧桐科苹婆属半落叶乔木植物...	猴面包树（学名：），又名-{zh-hant:猴面包树;zh-hans:猴树;zh-tw:猴面包树}-、非洲猴面包树、酸瓠树、猴树、旅人树、死老鼠树，是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲...
2	黄蝉（学名：'）别称为硬枝黄蝉、夹竹桃叶黄蝉、小花黄蝉、丛立黄蝉及黄兰蝉。为夹竹桃科黄蝉属植物...	黄蝉（学名：'）别称为硬枝黄蝉、夹竹桃叶黄蝉、小花黄蝉、丛立黄蝉及黄兰蝉。为夹竹桃科黄蝉属植物...
3	猴面包树（学名：），又名-{zh-hant:猴面包树;zh-hans:猴树;zh-tw:猴面包树}-、非洲猴面包树、酸瓠树、猴树、旅人树、死老鼠树，是一种锦葵科猴面包树属的大型落叶乔木，原产于热带非洲...	假苹婆（学名：" "），别称七姐果、鸡冠皮、鸡冠木、山羊角、红郎伞、赛苹婆及山木棉等，为梧桐科苹婆属半落叶乔木植物...

- ✓ 注意到余弦相似度召回结果（第一列）的顺序并不包含相似度排序信息，通常是按照其在节点组内的相对位置进行返回。
- ✓ 重排序得到的结果如第二列所示，可见重排序后与查询更相似的文档被排在了在更前面的位置。



目录



1. 上节回顾



2. RAG效果评测



3. 提升RAG系统的召回率



4. 综合优化策略的RAG



优化检索策略 – 多路召回



初始化在线大模型模块 (通义千问)

```
1 llm = lazyllm.OnlineChatModule(source='qwen', model='qwen-turbo')
```

加载文档数据并创建处理节点组

```
2 documents = lazyllm.Document(dataset_path="/mnt/lustre/share_data/dist/cmrc2018/data_kb")
```

创建分句处理节点组 (按句号切分)

```
3 documents.create_node_group(name='sentences', transform=lambda b: b.split('。'))
```

创建分段处理节点组 (按换行符切分)

```
4 documents.create_node_group(name='block', transform=lambda b: b.split('\n'))
```

创建基于段落块的检索器

```
5 retriever1 = lazyllm.Retriever(doc=documents, group_name="block", similarity="bm25_chinese", topk=3)
```

创建基于句子的检索器

```
6 retriever2 = lazyllm.Retriever(doc=documents, group_name="sentences", similarity="bm25_chinese", topk=3)
```

构建带上下文模板的prompt

```
7 prompt = '你是一个友好的 AI 问答助手，你需要根据给定的上下文和问题提供答案。 \n\n根据以下资料回答问题： \n\n{context_str} \n'
```

创建可复用的大模型对话接口

```
8 robot = llm.prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
```

执行问答流程

```
9 query = "亚硫酸盐有什么作用？"
```

双路检索获取相关段落和句子

```
10 doc_node_list1 = retriever1(query=query)
```

```
11 doc_node_list2 = retriever2(query=query)
```

合并多粒度检索结果

```
12 doc_node_list = doc_node_list1 + doc_node_list2
```



优化检索策略 - 多路召回



```
7_rag_multi_node_group.py — course [SSH: test0224]
7_rag_multi_node_group.py 1 X
courseware_codes > chapter7 > 7_rag_multi_node_group.py > ...
6
7 # 文档加载
8 documents = lazyllm.Document(dataset_path="/mnt/lustre/share_data/dist/cmrc2018/data_kb")
9 documents.create_node_group(name='sentences', transform=lambda b: b.split('。'))
10 documents.create_node_group(name='block', transform=lambda b: b.split('\n'))
11
12 # 检索组件定义
13 retriever1 = lazyllm.Retriever(doc=documents, group_name="block", similarity="bm25_chinese", topk=3)
14 retriever2 = lazyllm.Retriever(doc=documents, group_name="sentences", similarity="bm25_chinese", topk=3)
15
16 # prompt 设计
17 prompt = '你是一个友好的 AI 问答助手，你需要根据给定的上下文和问题提供答案。\'
18         根据以下资料回答问题：\'
19         {context_str} \'
20 robot = llm.prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
21
22 # 推理
23 query = "亚硫酸盐有什么作用？"
24 # 将Retriever组件召回的节点全部存储到列表doc_node_list中
25 doc_node_list1 = retriever1(query=query)
26 doc_node_list2 = retriever2(query=query)
27
28 # 将两个检索器的召回结果合并到一个列表
29 doc_node_list = doc_node_list1 + doc_node_list2
30
31 # 将query和召回节点中的内容组成dict，作为大模型的输入
32 res = robot({"query": query, "context_str": "".join([node.get_content() for node in doc_node_list])})
33
34 print("系统答案：", res)
35
```

问题 1 输出 调试控制台 终端 端口 19

o (lazyllm) → chapter7

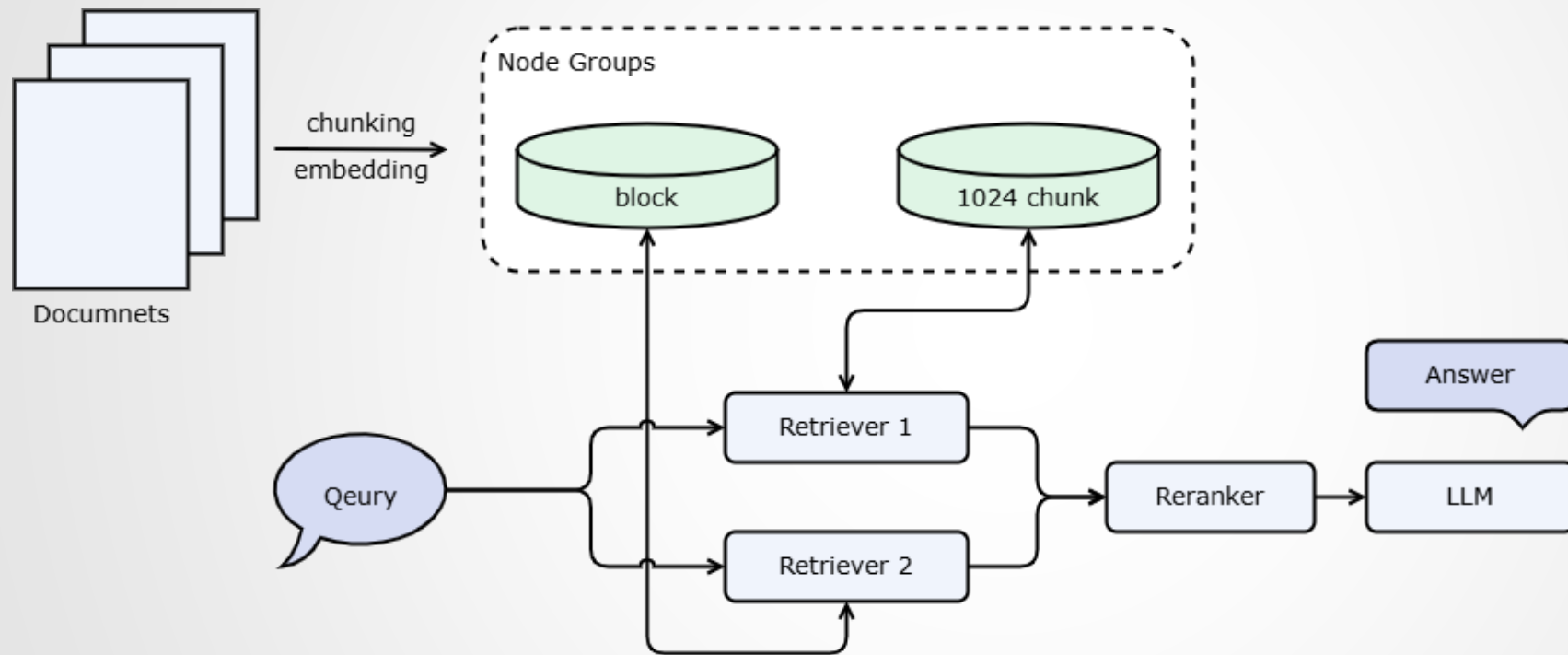
SSH: test0224 courseware_codes* 0 1 19

行 22, 列 5 空格: 4 UTF-8 LF Python 3.10.9 ('lazyllm': conda)



综合优化策略的RAG

综合上述策略，我们可以得到多路召回 RAG，其流程如下图所示，：



与朴素RAG相比有更好的检索效果，原因：

- ✓ 设置多个检索器在不同粒度进行检索或使用不同的相似度计算方法进行文档召回
- ✓ 对所有检索器召回的文档进行排序得到最终上下文。这是一种比较标准的多路检索RAG架构，



综合优化策略的RAG



初始化本地嵌入模型、本地重排序模型

```
1 embedding_model = lazyllm.TrainableModule("bge-large-zh-v1.5")
```

```
2 offline_rerank = lazyllm.TrainableModule('bge-reranker-large')
```

部署大语言模型(vLLM推理引擎)

```
3 llm = lazyllm.TrainableModule('internlm2-chat-20b', stream=True).deploy_method(lazyllm.deploy.Vllm)
```

加载文档并创建处理节点

```
4 docs = lazyllm.Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embedding_model)
```

```
5 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
```

定义问答提示模板

```
6 prompt = '你是一个友好的 AI 问答助手，你需要根据给定的上下文和问题提供答案。 \
```

```
7 根据以下资料回答问题： \
```

```
8 {context_str} \n '
```

构建端到端问答流水线

```
9 with lazyllm.pipeline() as ppl:
```

```
10     with lazyllm.parallel().sum as ppl.prl:
```

```
11         ppl.prl.retriever1 = lazyllm.Retriever(doc=docs, group_name="CoarseChunk", similarity="cosine", topk=3)
```

```
12         ppl.prl.retriever2 = lazyllm.Retriever(doc=docs, group_name="block", similarity="bm25_chinese", topk=3)
```

```
13
```

```
14     ppl.reranker = lazyllm.Reranker(name='ModuleReranker', model=offline_rerank, topk=3) | bind(query=ppl.input)
```

```
15
```

```
16     ppl.formatter = (lambda nodes, query: dict(
```

```
17         context_str= "" .join([node.get_content() for node in nodes]), query=query)
```

```
21     ) | bind(query=ppl.input)
```

```
22
```

```
23 ppl.llm = llm.prompt(lazyllm.ChatPrompinstruction=prompt, extra_keys=['context_str']))
```

```
24
```

```
25 ppl.start()
```



综合优化策略的RAG



13_rag_use_with_statement.py — course [SSH: test0224]

13_rag_use_with_statement.py 2 X

courseware_codes > chapter7 > 13_rag_use_with_statement.py > ...

```
1 import lazyllm
2 from lazyllm import bind
3
4 # 定义嵌入模型和重排序模型
5 # online_embedding = lazyllm.OnlineEmbeddingModule()
6 embedding_model = lazyllm.TrainableModule("bge-large-zh-v1.5").start()
7
8 # 如果您要使用在线重排模型
9 # 目前LazyLLM仅支持 qwen和glm 在线重排模型, 请指定相应的 API key。
10 # online_rerank = lazyllm.OnlineEmbeddingModule(type="rerank")
11 # 本地重排序模型
12 offline_rerank = lazyllm.TrainableModule('bge-reranker-large').start()
13
14 llm = lazyllm.TrainableModule('internlm2-chat-20b', stream=True).deploy_method(lazyllm.deploy.Vllm).start()
15
16 docs = lazyllm.Document("/mnt/lustre/share_data/dist/cmrc2018/data_kb", embed=embedding_model)
17 docs.create_node_group(name='block', transform=(lambda d: d.split('\n')))
18 prompt = '你是一个友好的 AI 问答助手, 你需要根据给定的上下文和问题提供答案。\'
19         根据以下资料回答问题: \'
20         {context_str} \'
21
22 with lazyllm.pipeline() as ppl:
23     # 在 block 节点组执行两种相似度的检索召回, lazyllm.parallel 并行执行两个检索操作
24     with lazyllm.parallel().sum as ppl.prl:
25         ppl.prl.retriever1 = lazyllm.Retriever(doc=docs, group_name="CoarseChunk", similarity="cosine", topk=3)
26         ppl.prl.retriever2 = lazyllm.Retriever(doc=docs, group_name="block", similarity="bm25_chinese", topk=3)
27
28     # 对上述两个检索器召回的文档进行重排序
29     ppl.reranker = lazyllm.Reranker(name='ModuleReranker', model=offline_rerank, topk=3) | bind(query=ppl.input)
30
31     ppl.formatter = (
32         lambda nodes, query: dict(
```

问题 2

输出

调试控制台

终端

端口 9

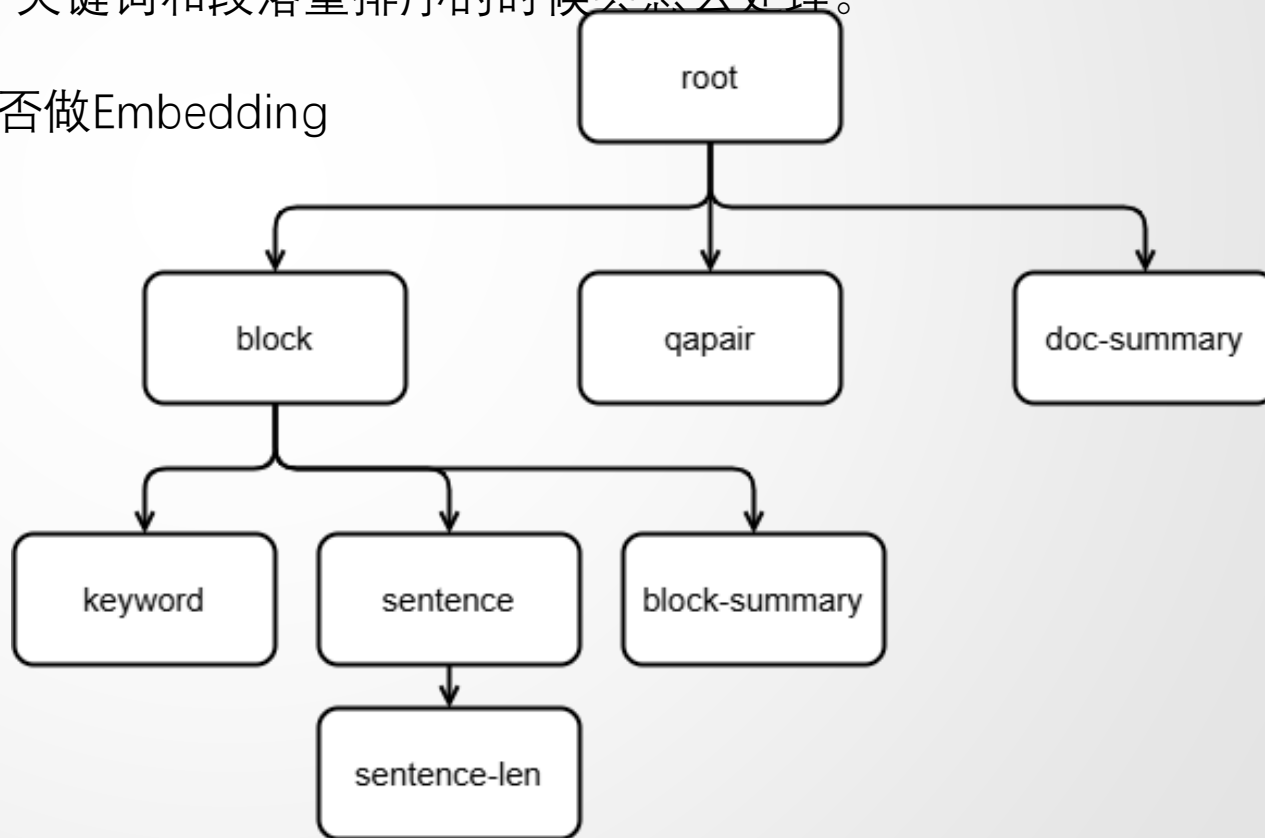
+ zsh - chapter7

o (lazyllm) → chapter7



Q&A

1. 如果我同时使用关键词和段落做召回，关键词和段落重排序的时候会怎么处理。
2. QA（问答对）或 summary（总结）是否做Embedding



感谢聆听
Thanks for Listening

